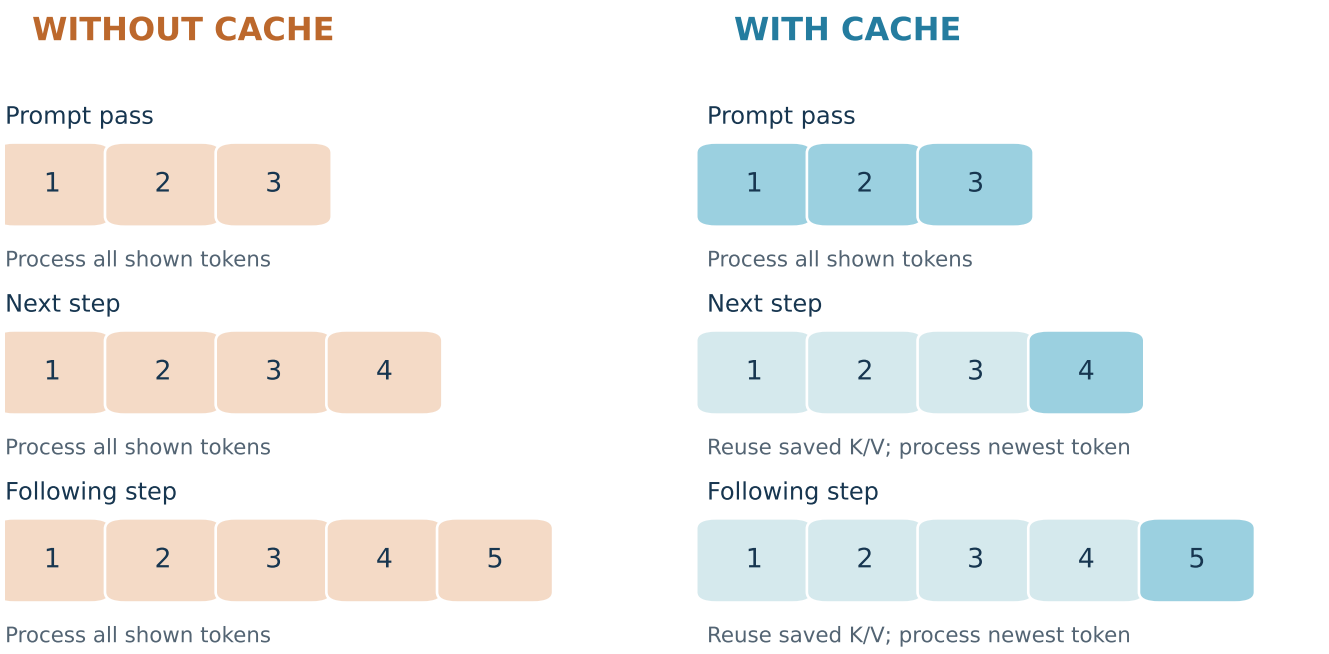
What KV caching changes

Same weights and tested outputs; less repeated computation

A three-token prompt is followed by two decoding steps. Orange and blue boxes are processed now; pale boxes represent reused K/V. Both paths still attend to the preceding history.



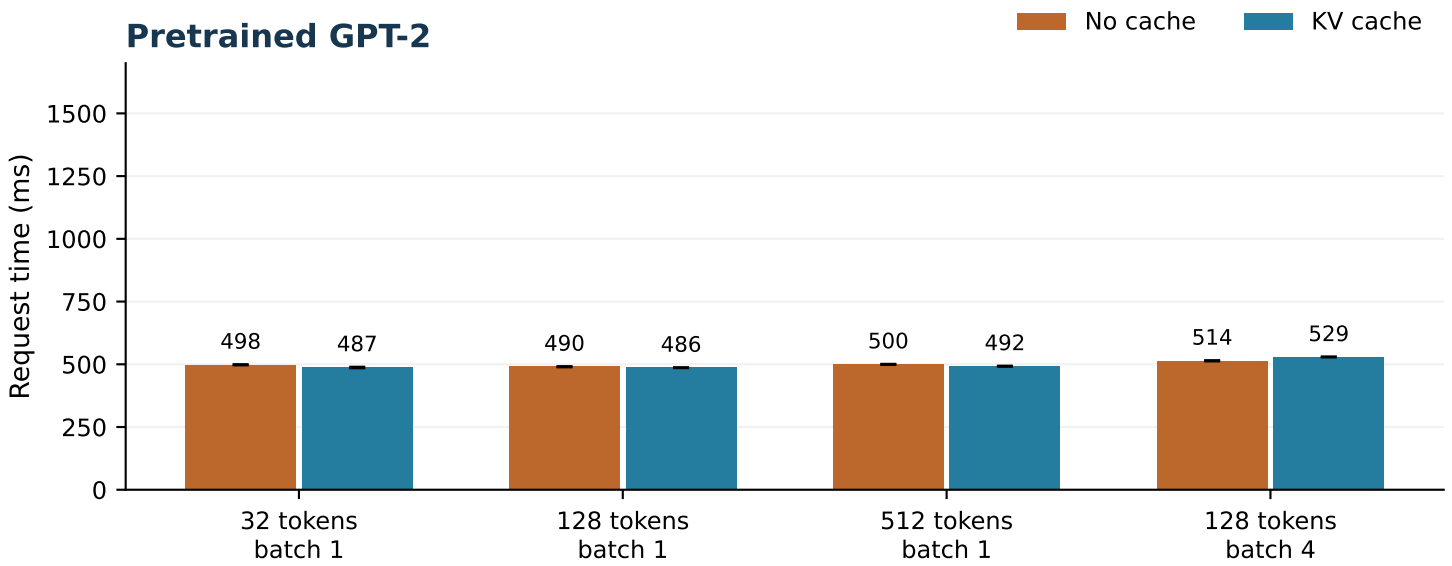
Within each attention layer, the new token supplies a query. That query attends to saved keys and values plus the new token's own key and value. The updated K/V are kept for the next step.

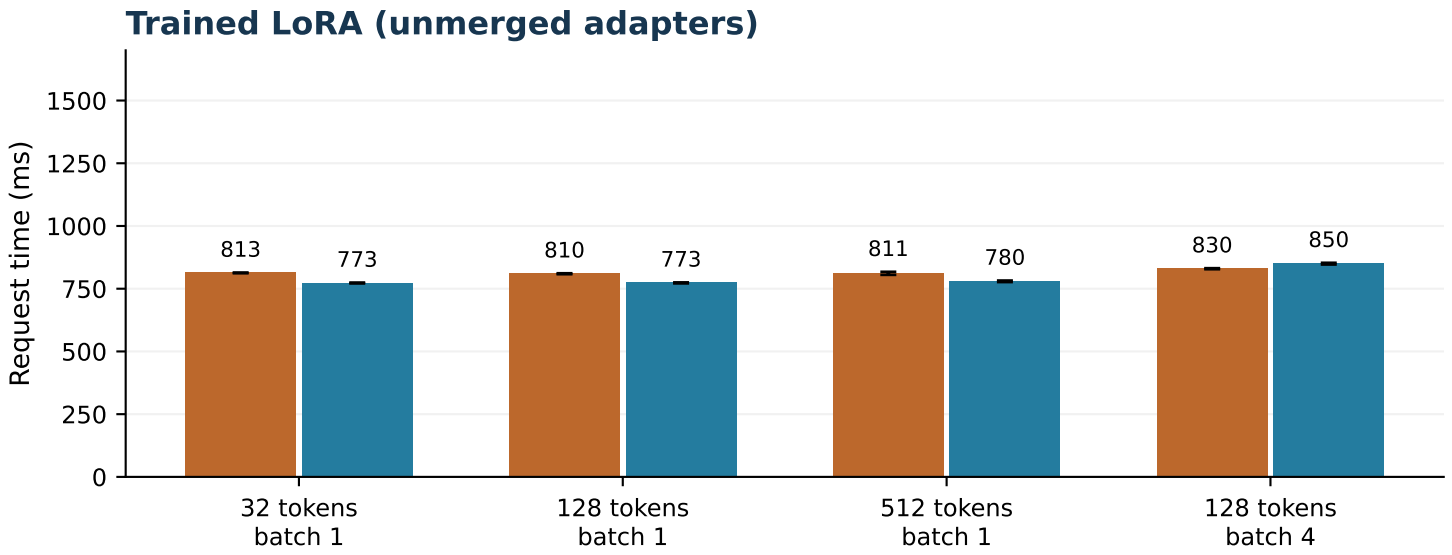


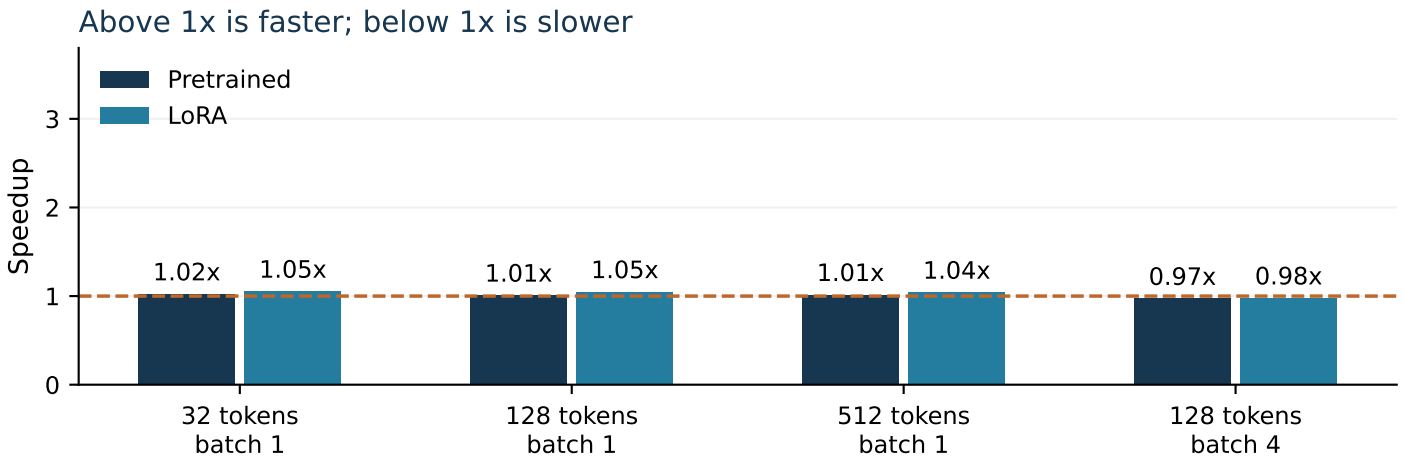
Each of the 12 layers has its own cache. Caching adds no learned parameters. At the 1,024-token context limit, this implementation rebuilds the cropped window to preserve minGPT's learned-position behavior; incremental speed benefits then stop.

Where caching speeds up generation

NVIDIA A30 | FP32 | 64 generated tokens per sequence | 5 measured repeats



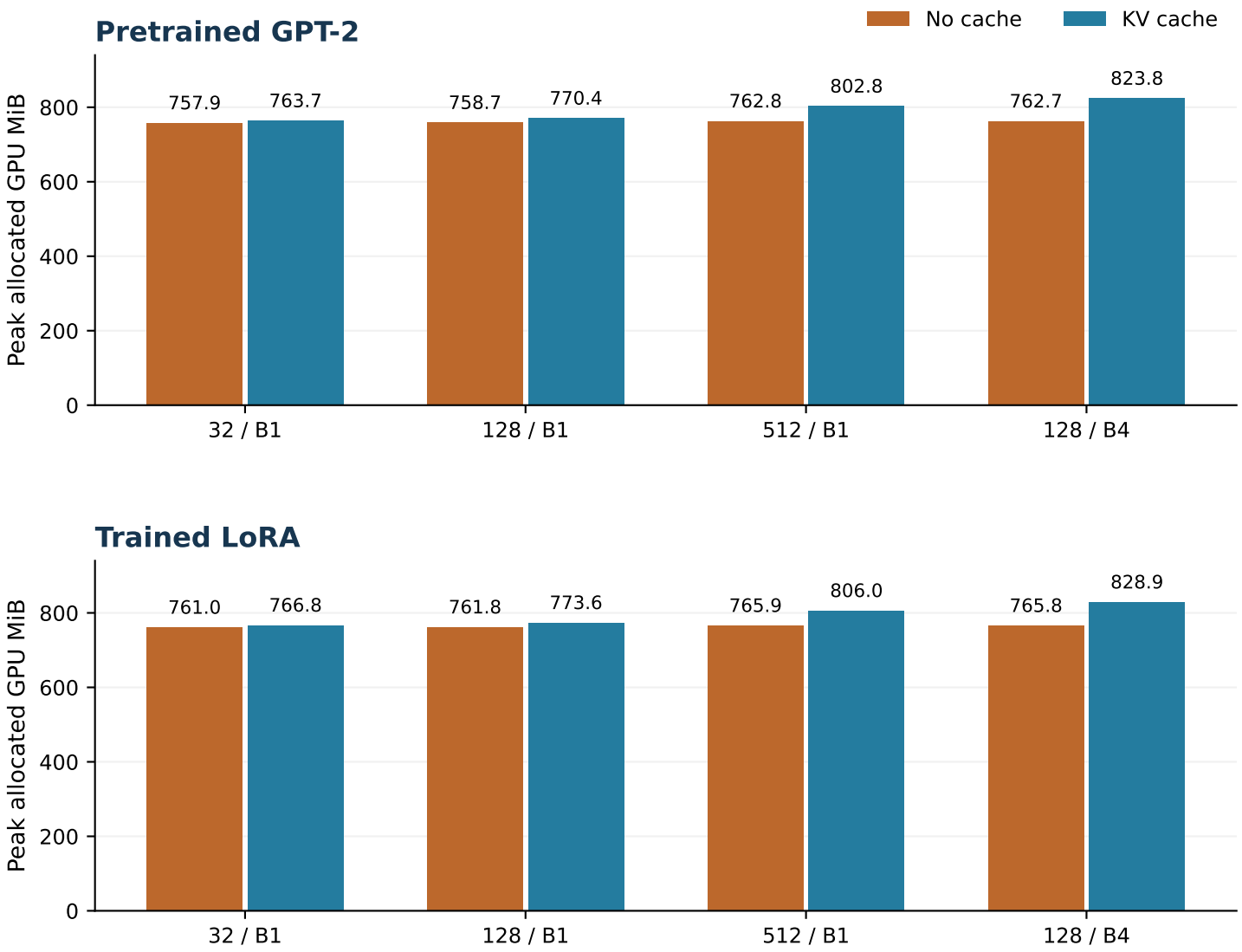




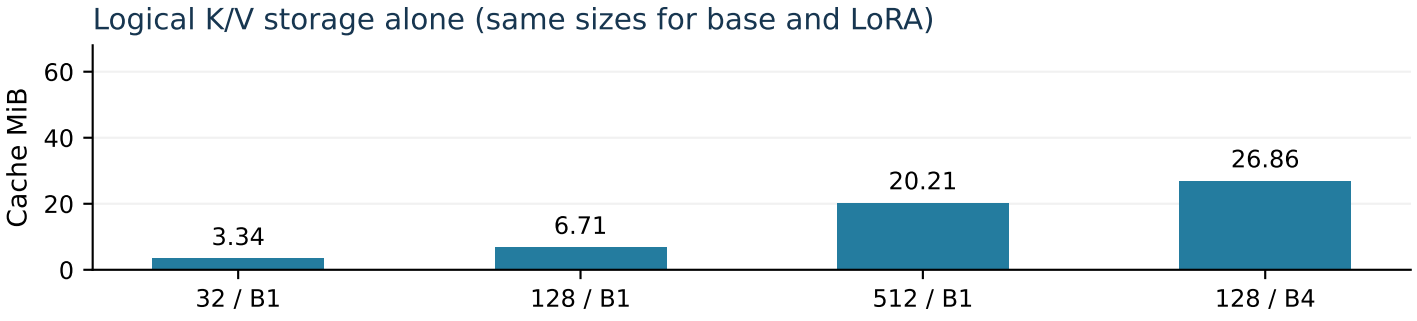
Longer prompts and batch four benefit most; short requests are near break-even. Error bars show one standard deviation. Timings exclude model loading and tokenization.

The speed–memory trade-off

Cache storage is temporary inference state, not additional model weights



X-axis: prompt tokens / batch size. Memory axes start at zero. Peak allocation includes weights, masks, caches, and live temporaries. It is measured runtime memory, not simply the sum of parameter and cache storage.



The cache requires 72 KiB per stored token per sequence in FP32. After predicting 64 output tokens, it holds prompt length + 63 tokens. Cache storage grows with sequence length and batch size; it is already included in the peak-memory measurements above.

KV caching: implementation and validation

MQP | FP32 inference baseline | KV caching versus no cache

Purpose

Reduce repeated work during autoregressive generation while preserving model outputs. This is an inference optimization: no training, extra learned parameters, or weight updates are required.

How it works in this minGPT

The first forward pass processes the full prompt (prefill). Each of the 12 transformer layers saves its attention keys and values. Subsequent steps process only one new token; its query attends to the saved history. Cached length offsets both the causal mask and learned position embeddings. The cache grows as generation proceeds.

Implementation

Opt-in API: model.eval(); model.generate(ids, 64, use_cache=True). Direct forward calls accept past_key_values and return a per-layer cache. Existing training and checkpoint formats remain compatible. Both benchmark modes compute only final-position output logits, avoiding a separate output-head optimization as a confound.

Validation on real checkpoints

Checkpoint	Identical outputs	Accuracy: off / on	Max logit difference
pretrained	99/120	24.2% / 22.5%	0.250000
full	120/120	18.3% / 18.3%	0.078125
lora	96/120	23.3% / 23.3%	0.250000

Meaning of the checks

All 40 generated token IDs matched for every one of the 120 held-out sports prompts in each checkpoint: 360/360 comparisons. These are seed-42 checkpoints; accuracy here is not a three-seed average. Chunked FP32 logits passed combined absolute/relative tolerances of 0.0002 and had identical argmax tokens. Before/after parameter hashes matched for every model.

Additional tests

All 18 repository tests passed, including four KV-cache tests. Coverage includes chunked decoding, nonzero LoRA adapters, batched generation, seeded sampling, context-window overflow, independent requests, normal training, and checkpoint compatibility. The CPU demo also reloads the trained adapter and compares both generation paths.

Source: experiments/kv_cache/results/report.json and saved per-model outputs.

Inference speed: measured results

MQP | FP32 inference baseline | KV caching versus no cache

Configuration

NVIDIA A100-SXM4-80GB; FP32; TF32 off; greedy decoding; 64 new tokens per sequence. Two warmups and five measured repetitions per mode/configuration, with alternating mode order. Prompts contain deterministic random token IDs to control lengths. Timings exclude model loading, tokenization, input transfer, and warmups. Job 2408911 on gpu-4-15.int.turing.wpi.edu.

Pretrained GPT-2: request time and aggregate throughput

Batch; prompt	No cache (ms)	Cache (ms)	Speedup	Tokens/s: off / on
1; 32	498.0 +/- 1.2	487.2 +/- 1.9	1.02x	128.5 / 131.4
1; 128	490.2 +/- 1.2	486.4 +/- 0.6	1.01x	130.5 / 131.6
1; 512	499.6 +/- 1.1	492.4 +/- 1.0	1.01x	128.1 / 130.0
4; 128	513.9 +/- 1.4	529.3 +/- 0.2	0.97x	498.1 / 483.7

Trained LoRA: request time and aggregate throughput

Batch; prompt	No cache (ms)	Cache (ms)	Speedup	Tokens/s: off / on
1; 32	813.1 +/- 1.4	772.8 +/- 1.8	1.05x	78.7 / 82.8
1; 128	809.7 +/- 1.9	773.2 +/- 2.0	1.05x	79.0 / 82.8
1; 512	811.3 +/- 6.1	779.6 +/- 3.1	1.04x	78.9 / 82.1
4; 128	829.7 +/- 2.1	849.9 +/- 3.5	0.98x	308.5 / 301.2

Reading these tables

Batch is the number of sequences generated together; prompt is tokens per sequence. Request time is synchronized wall time to generate all 64 tokens per sequence. +/- is standard deviation across five timing repetitions, not a confidence interval. Throughput counts output tokens across the whole batch. These are preliminary measurements on one allocated GPU.

Interpretation

Caching provides its strongest measured benefit for longer prompts and larger batches. The shortest batch-one case is approximately break-even or slightly slower, so caching is not a universal speedup. Small requests offer less repeated work to eliminate while still paying cache management and execution overhead. This explanation is an inference from scaling behavior, not a kernel-level bottleneck diagnosis.

Scope of the comparison

Both pretrained and LoRA architectures are benchmarked. Full fine-tuning uses the same inference architecture as pretrained GPT-2; its outputs were verified, but no separate full-checkpoint timing claim is made. LoRA adapters remain unmerged. There is no FlashAttention, mixed precision, compilation, or serving scheduler.

Source: experiments/kv_cache/results/report.json and saved per-model outputs.

Prompt latency, decode latency, and GPU memory

MQP | FP32 inference baseline | KV caching versus no cache

Pretrained GPT-2: prefill and decode

Batch; prompt	Prefill: off / on (ms)	Decode: off / on (ms/token)
1; 32	9.21 / 9.15	7.76 / 7.59
1; 128	9.19 / 9.93	7.63 / 7.56
1; 512	9.03 / 9.69	7.79 / 7.66
4; 128	9.48 / 9.78	8.01 / 8.24

Trained LoRA: prefill and decode

Batch; prompt	Prefill: off / on (ms)	Decode: off / on (ms/token)
1; 32	14.25 / 14.46	12.68 / 12.04
1; 128	14.37 / 15.14	12.62 / 12.03
1; 512	14.56 / 15.06	12.65 / 12.13
4; 128	14.83 / 15.23	12.93 / 13.25

Timing definitions

CUDA-event prefill time includes processing the prompt and choosing/appending the first output token. Decode latency is the subsequent 63 steps divided by 63; for batch four it is latency per batched token step, not per individual sequence token. Caching primarily reduces subsequent decode work, not initial prompt processing.

Peak allocated GPU memory and logical cache storage (MiB)

Model; batch; prompt	No cache peak	Cache peak	KV storage
pretrained; 1; 32	757.87	763.69	3.34
pretrained; 1; 128	758.71	770.44	6.71
pretrained; 1; 512	762.79	802.83	20.21
pretrained; 4; 128	762.66	823.77	26.86
lora; 1; 32	760.99	766.81	3.34
lora; 1; 128	761.84	773.56	6.71
lora; 1; 512	765.92	805.96	20.21
lora; 4; 128	765.78	828.92	26.86

Memory interpretation

Peak allocated memory includes model weights, attention masks, inputs, caches, and live temporary tensors. Reserved-memory samples are also saved in JSON/CSV; do not add reserved and allocated values. Logical FP32 cache size is 2 x layers x batch x cached_tokens x hidden_width x 4 bytes: 72 KiB per token per sequence here. Dynamic concatenation can temporarily retain both old and new cache tensors.

Source: experiments/kv_cache/results/report.json and saved per-model outputs.

Demonstration, limitations, and reproduction

MQP | FP32 inference baseline | KV caching versus no cache

A real held-out example

Prompt: Bulls vs Sharks. Halftime 51-35; final 96-69. Who won? The trained LoRA model answers "Bulls won 96-69." with caching off and on. The fully fine-tuned model retains its incorrect "Rockets won 96-69." answer in both modes. Caching preserves the existing model behavior; it does not improve reasoning or correct factual mistakes.

The context-window boundary

This GPT-2 uses 1,024 learned absolute positions. Once the window fills, existing minGPT crops old tokens and resets positions. Dropping only the oldest cached K/V would not preserve that behavior. Cached generate therefore rebuilds from the cropped window at saturation. Correctness is preserved, but the incremental speed benefit is lost after saturation. All measured performance cases stay within the context window.

Other practical limits

Caches require evaluation mode and disabled autograd. Use equal-length batches; padding masks and per-sequence cache lengths are not implemented. Discard a cache when changing the prompt, weights, or LoRA adapter settings. Five timing repeats and a synthetic task do not establish production serving performance.

Try the models

From ~/MQP/MinGPT/minGPT after sourcing ~/MQP/Torch/setup.sh: python kv_cache_demo.py --model lora --device cpu. Use --model full or --model pretrained for other checkpoints. On an allocated GPU use --device cuda. The demo prints both answers and whether generated tokens match; its single-request timings are illustrative.

Reproduce the experiment

Run: sbatch experiments/kv_cache/run_gpu.sh. Focused tests: python -m unittest discover -s tests -p test_kv_cache.py -v. Saved report.json includes raw timing samples, environment details, source hashes, before/after parameter hashes, and checkpoint paths. The separate *-quality.json files contain every cached/uncached answer and score.

Finding for the meeting

KV caching is implemented and validated for base, fully fine-tuned, and LoRA models. It preserved all tested generated outputs and all model weights. It accelerated larger inference workloads while increasing measured peak GPU memory in these tests. The short-prompt and context-limit cases show where the benefit is limited.

Source: experiments/kv_cache/results/report.json and saved per-model outputs.